# Project Hail Mary - Rescue

## First project of languages and programming environments 25/26

26 March 2026 (Due date, 12 April 2026)

## Background

*Project Hail Mary* is a 2021 science fiction novel by Andy Weir — the acclaimed author of *The Martian* — now playing in IMAX theaters. It follows Dr. Ryland Grace, a lone astronaut who finds himself on a desperate mission far from Earth, tasked with finding a way to save humanity from extinction.

**Spoiler alert: if you haven't read the book or seen the movie, jump to section Problem Statement — or better yet, go see it first!**

The story begins with Ryland waking up alone aboard a spacecraft with no memory of who he is or where he is. He woke up from an induced coma, and his two crewmates are dead, reduced to mummified remains. His own memories return only in fragmented flashbacks. As those memories surface, he pieces together a terrifying picture: an alien microorganism called Astrophage is draining the Sun's energy, threatening all life on Earth with extinction within decades. Earth scientists discovered that Tau Ceti, a distant star, was unaffected by Astrophage, and Ryland's mission is to go there, find the cause for such an anomaly, and transmit whatever solution he could find back to Earth via small autonomous probes.

The bulk of the story occurs when Ryland arrives at Tau Ceti and encounters an alien spacecraft and makes contact with its lone occupant — an intelligent being from the planet Erid, whom Ryland nicknames Rocky, travelling in a ship named, by Ryland, the *Blip-A*.

© *Amazon Studios*

Despite their significant biological differences — Rocky breathes ammonia, perceives the world through echolocation, and lives under immense atmospheric pressure — the two form an unexpected friendship and agree to cooperate to save both their homeworlds.

In the final act, aboard the crippled *Hail Mary*, Ryland must race through the ship's damaged internal corridor to reach the **Blip-A drive module** before a spreading contamination threatens the ship. The path is treacherous: decompressed sections threaten instant death, radiation leaks from ruptured shielding, and clusters of Astrophage that can consume a human being's body heat in minutes. Fortunately, Rocky, a great engineer, managed to scatter a few emergency tools through the safer sections before the contamination cut off access. These tools are a **Taumoeba vial**, a **thermal insulation wrap**, and an **EVA suit**.

---

## Problem Statement

The problem is to determine the minimum time Ryland needs to traverse the corridor from his starting point to the drive module, given the layout of hazards and tools along the way. The corridor is a sequence of **sections**. Each section contains a single hazard or none. Tools are found in sections that do not have any hazard, called **safe sections**. Each safe section has a single tool or none.

At any moment, Ryland can carry **at most one tool**. When he arrives at a safe section, he can keep or drop the tool he entered with (if he entered with one), and he can ignore or pick up the tool in the section (if there is one), as long as he never carries more than one tool at a time.

It takes Ryland **1 time unit** to cross a safe section if he enters and exits with no tool. Time goes up **1 unit** if he enters a section with a tool, and up **1 unit** if he leaves a section with a tool. Therefore, it may take 1, 2, or 3 time units to cross a safe section. The possible tools are the following, each with a different cost to cross a hazard:

- **Taumoeba vial** — works only against Astrophage clusters; the microorganisms consume the Astrophage on contact. Costs **4 time units** through an Astrophage section.
- **Thermal insulation wrap** — shields against Astrophage's heat drain or blocks a radiation zone, but offers no protection against vacuum. Costs **5 time units** through a Radiation or Astrophage section.
- **EVA suit** — allows passage through any hazard (Astrophage, Radiation, or Vacuum). Costs **6 time units** through any hazardous section.

Tools are **not consumed** after use — Ryland keeps a tool until he deliberately drops it. It is guaranteed that, for all given inputs, Ryland can always reach the drive module.

## Task

Write two functions in OCaml that, 1. given the layout of the corridor, returns a list of values representing the corridor sections in order; 2. given such a list, returns the minimum number of time units Ryland needs to traverse the corridor. For the inputs given it is always possible to find the solution.

The corridor is encoded as a string where each character represents one section:

| Character | Section |
|---|---|
| `_` | Safe section with no tool |
| `T` | Safe section with a Taumoeba vial |
| `W` | Safe section with a Thermal insulation wrap |
| `E` | Safe section with an EVA suit |
| `A` | Astrophage cluster |
| `R` | Radiation zone |
| `V` | Vacuum breach |

## Signatures

We provide a `dune` project with general signatures (in file `hailmary.mli`) for the two functions you need to implement. In your implementation (file `hailmary.ml`), you should define the type `section` for the sections of the corridor, and implement the two functions `parse_corridor` and `min_time` according to the specifications above.

```
type section
val parse_corridor : string -> section list
```

```
val min_time : section list -> int
```

You should provide purely functional definitions, not using any kind of mutable state. You can define any helper functions you need, but the two functions above are the only ones that will be tested.

## Examples

**Example 1** — expected output: `34`

```
assert (min_time (parse_corridor "W__ERRVTWA_") = 34)
```

Strategy: ignore the first wrap (`W`), pick the EVA suit (`E`), use it through both radiation zones and the vacuum (`RR V`), drop it at the Taumoeba section (`T`), pick the second wrap (`W`), use it through the Astrophage cluster (`A`), drop it at the last section. Time: 1+1+1+2+6+6+6+2+2+5+2 = 34.

**Example 2** — expected output: `39`

```
assert (min_time (parse_corridor "_E_V______V__") = 39)
```

Strategy: pick the EVA suit (`E`) at section 2 and carry it through both vacuum sections (`V...V`), dropping it at the penultimate section. Time: 1+2+3+6+(6×3)+6+2+1 = 39.

## Submission details

The project will be submitted via autolab, and will be tested against a hidden test suite. You can run the public tests with `dune test`. The autolab assignment will be available at (`https:://lap.di.fct.unl.pt`) until **11:59 PM on April 12, 2026**. You can submit as many times as you want before the deadline, but only your last submission will be graded.

## Rules

This assignment must be completed individually. You are allowed to discuss ideas with your classmates and ask questions to the instructors and in discord, but you are not allowed to share code with anyone else. Please refer to the course rules online and the NOVA FCT Knowledge Assessment Regulation for more details on what is considered fraud and plagiarism, and the consequences of such actions. Students who commit fraud on an assignment will not receive frequency for it.

## Evaluation criteria

Your solution will be evaluated based on the following criteria:

- **Correctness**: your code should return the correct output for all test cases, including hidden ones. This will be the most important criterion, and will be worth 80% of the grade.

- **Code quality**: your code should purely functional, be well-structured, readable, and follow good programming practices. This includes using meaningful variable names, writing clear and concise code, and avoiding unnecessary complexity. This will be worth 20% of the grade.